



南開大學
Nankai University

计算机学院
并行程序设计课程报告

SIMD 架构编程: ANN

姓名：黄泽恺

学号：2411268

专业：计算机科学与技术

2026 年 5 月 8 日

目录

1 摘要	2
2 实验平台信息与源码链接	2
3 实验问题描述	2
3.1 ANNS 向量检索问题	2
3.2 评价指标	2
4 Flat-SIMD	3
4.1 程序实现思路	3
4.2 实验结果呈现及分析	4
5 SQ-SIMD	4
5.1 程序实现思路	4
5.2 程序核心优化	6
5.3 实验结果呈现及分析	7
6 PQ-SIMD	8
6.1 程序实现思路	8
6.2 程序核心优化	9
6.3 实验结果呈现及分析	12
7 汇编代码分析	13
8 实验总结与思考	14

1 摘要

本次实验围绕近似最近邻搜索 (Approximate Nearest Neighbor Search, ANNS) 中的向量检索任务展开, 重点研究在大规模高维向量数据集上如何通过算法优化与 SIMD 并行化技术降低查询延迟并保持较高的检索准确率。实验首先实现了基于原始向量逐点计算的 Flat 检索方法, 并在此基础上利用 ARM NEON SIMD 指令对内积或 L2 距离计算过程进行向量化加速, 从而提升基础检索的计算效率。随后, 实验进一步引入标量量化 (Scalar Quantization, SQ) 与乘积量化 (Product Quantization, PQ) 方法, 通过压缩向量表示、构建查找表以及采用粗排加精排的两阶段检索流程, 减少检索过程中的计算量与访存开销。

在实验过程中, 本文对不同检索方法的 latency 和 recall 进行了测试与分析, 并重点探讨了 Top-p 大小、SIMD 指令优化等因素对检索性能的影响, 并借助 perf 工具深入 profiling 程序的性能表现, 分析内在原因。实验结果表明, Flat-SIMD 能够显著提升原始向量距离计算效率, SQ 和 PQ 方法能够降低粗排阶段的计算与存储成本, 但其最终效果受到量化精度、候选集规模和精排策略的影响较大。通过合理设置 Top-p、采用原始向量精排以及优化查表累加过程, 可以在 latency 和 recall 之间取得较好的 trade-off。本实验加深了对向量检索算法、量化压缩方法以及 SIMD 数据级并行优化技术的理解, 也为后续在实际高维检索系统中进行性能调优提供了实践基础。

2 实验平台信息与源码链接

本次实验在 OpenEuler 服务器上进行。

本次实验已上传 Github https://github.com/Redamancykai/NKU_Parallel/tree/main/Lab2

3 实验问题描述

3.1 ANNS 向量检索问题

近似最近邻搜索 (Approximate Nearest Neighbor Search, ANNS) 是高维向量检索中的核心问题。给定一个包含 N 条 d 维向量的数据库 $X = \{x_1, x_2, \dots, x_N\}$, $x_i \in \mathbb{R}^d$, 以及一个查询向量 $q \in \mathbb{R}^d$, 最近邻搜索的目标是在数据库中找到与 q 距离最近的 Top- k 个向量。

本次实验我们重点针对 ANNS 算法的查询部分进行优化, 主要从 SIMD 指令优化和 VQ 算法优化两方面出发。

3.2 评价指标

本实验主要采用 latency 和 recall 两个指标评价检索方法的性能。

Latency 表示单个查询或一批查询的平均查询延迟, 反映算法的实际运行效率。

Recall@k 用于衡量近似检索结果与精确检索结果之间的一致程度, 定义为

$$\text{Recall@k} = \frac{|KNN(q) \cap KANN(q)|}{k},$$

其中 $KNN(q)$ 表示精确搜索得到的结果集合, $KANN(q)$ 表示使用 ANNS 得到的结果集合。

4 Flat-SIMD

4.1 程序实现思路

Flat 方法是最直接的向量检索方法。对于每一个查询向量 q ，程序遍历数据库中的所有 base 向量 x_i ，分别计算 q 与 x_i 之间的距离或相似度，然后维护一个 Top- k 结果集合。该方法是 SIMD 后续优化的基础，是后续 SQ 和 PQ 方法进行精排的基准。

在原始串行版本中，距离计算需要对向量的每一个维度依次进行乘加或差平方累加。例如内积计算的核心循环为

$$sum = \sum_{j=0}^{d-1} q_j x_j.$$

该过程具有明显的数据级并行特征：每一维的乘法之间相互独立，最终只需要进行归约求和。因此可以使用 ARM NEON SIMD 指令对内层循环进行向量化，使得一次指令同时处理多个 float 数据。由于 Flat 方法需要对每个查询遍历全部 base 向量，因此距离计算函数会被反复调用，我们重点对内积距离计算函数进行优化：

```

1 float InnerProductSIMDNeon_Unroll32(const float* b1, const float* b2, size_t vecdim) {
2     assert(vecdim % 32 == 0);
3     // 定义 4 个 128 位累加器进行 4 路累加
4     simd8float32 sum0(0.0);
5     simd8float32 sum1(0.0);
6     simd8float32 sum2(0.0);
7     simd8float32 sum3(0.0);
8     // 32 位循环展开，每次处理 32 个 float
9     for(size_t i = 0; i < vecdim; i += 32) {
10        // 加载 b1, b2 的前 8 个 float，使用 fma 乘加指令累加到 sum0
11        simd8float32 r0(b1 + i);
12        simd8float32 s0(b2 + i);
13        sum0.fmadd(r0, s0);
14        // 以此类推处理剩余 3 组
15        ...
16    }
17    // 合并规约
18    sum0 += sum1;
19    sum2 += sum3;
20    sum0 += sum2;
21    // vaddvq_f32 指令对所有 float 元素求和
22    float dot = vaddvq_f32(sum0.data.val[0]) + vaddvq_f32(sum0.data.val[1]);
23    return 1 - dot;
24 }
```

首先，程序使用 NEON 向量加载指令一次读取多个连续 float 数据。相比普通串行循环每次只处理一个维度，SIMD 版本可以在一个循环迭代中处理 4 个维度，减少循环控制开销，并提升数据级并行度。其次，程序使用向量乘加 FMA 指令优化乘法和加法组合，FMA 可以将乘法和加法融合为一类指令，在减少指令数量的同时提高流水线利用率。最后，程序通过循环展开进一步降低循环分支开销。在 InnerProductSIMDNeon_Unroll32 中，循环体每轮处理 32 个 float，同时使用 4 个 NEON 向量寄存器进行累加，增强了程序的并行执行能力，使 CPU 有更多机会进行指令级并行调度。

4.2 实验结果呈现及分析

针对 Flat Search, 我们重点比较初始未作任何优化的 Flat 串行版本、Flat-SIMD 基础版本、Flat-SIMD-FMA、Flat-SIMD-Unroll 版本之间的性能。重点比较不同方法的 recall 和 latency 差异, 同时使用 perf 分析 Flat 串行版本和我们最终优化的 Flat-SIMD-Unroll 的 cycles、instructions 和 cache misses 等指标。

表 1: Flat 方法不同实现的性能对比

方法	latency / us	recall	加速比
Flat-Naive	17497.8	0.99995	-
Flat-SIMD	6619.94	0.99995	2.64
Flat-SIMD-FMA	5742.21	0.99995	3.05
Flat-SIMD-Unroll	5623.78	0.99995	3.11

可以看到, 几个版本的 average recall 都是 0.99995, 近似于 1, 说明 SIMD 优化没有改变检索结果的正确性, 优化后的程序仍然能够保持和原始串行版本几乎完全一致的召回率。同时在仅将内积计算向量化的优化下 (Flat-SIMD), 程序就获得了 2.64 倍的加速, 可见程序的整体耗时主要集中在距离计算上。在此基础上, 我进一步使用 FMA 乘加指令优化向量计算, 并通过循环展开进一步控制开销, 最终相比初始版本达到了 3.11 倍的加速。为了进一步分析原因, 我使用 perf 分析了 Flat-Naive 和 Flat-SIMD-Unroll 两个版本:

表 2: Flat-Naive 和 Flat-SIMD-Unroll 的性能对比

方法	cycles	instructions	cache-references	cache-misses
Flat-Naive	90156867167	118270999252	38834098348	499759194
Flat-SIMD-Unroll	35119578104	19671108562	10033398146	170345304

根据 perf 统计结果, Flat-SIMD 优化后程序的 cycles 从 9.02×10^{10} 降低到 3.51×10^{10} , 减少约 61.1%; instructions 从 1.18×10^{11} 降低到 1.97×10^{10} , 减少约 83.4%。

从缓存行为来看, Flat-SIMD 的 cache-references 从 3.88×10^{10} 降低到 1.00×10^{10} , cache-misses 从 4.99×10^8 降低到 1.70×10^8 , 说明向量化减少了循环访存次数和缓存访问压力。虽然 cache miss rate 从 1.287% 上升到 1.698%, 但由于总访问次数大幅减少, cache miss 的绝对数量仍然明显下降, 因此总体缓存开销降低。

综上, Flat-SIMD 通过 ARM NEON 向量化内积计算, 显著减少了指令数和 CPU 周期数, 使平均查询延迟大幅下降, 实现约 3 倍加速, 并保持了 0.99995 的高召回率。实验结果表明, SIMD 向量化是 Flat 检索非常有效的性能优化手段。

5 SQ-SIMD

5.1 程序实现思路

SQ (Scalar Quantization, 标量量化) 是在 Flat 方法基础上的进一步优化。Flat 方法中, 每个向量维度通常使用 32-bit float 存储和计算。SQ 的核心思想是将每一维浮点数映射为低精度整数 (0-255), 从而减少向量存储空间和访存带宽需求。对于 SIMD 而言, 低精度整数还可以提高单条向量指令同时处理的数据个数。

我们首先要做的是将 base 向量量化到 `uint8_t`，主要依据公式

$$x_q = \text{round}\left(\frac{x - \min}{\max - \min} \times 255\right).$$

```

1 static inline uint8_t quantize_float_to_u8(float x, float min_val, float inv_step) {
2     int q = static_cast<int>(std::round((x - min_val) * inv_step));
3     if (q < 0) q = 0;
4     if (q > 255) q = 255;
5     return static_cast<uint8_t>(q);
6 }

```

本实验中的 SQ-SIMD 采用两阶段检索流程。第一阶段在量化空间中进行粗排，快速筛选出 Top- p 个候选向量；第二阶段在原始浮点空间中对这 p 个候选进行精排，最终得到 Top- k 检索结果。SQ-SIMD 的整体查询流程如下：

Algorithm 1 SQ-SIMD 两阶段查询流程

Input: 查询向量 q ，SQ 编码后的数据库 X_q ，原始数据库 X ，候选数量 p ，返回数量 k

Output: Top- k 最近邻结果

- 1: 量化 base 向量
 - 2: 将查询向量 q 量化为 q_q
 - 3: 量化空间粗排得到候选集合 C
 - 4: 复用 Flat-SIMD 精排得到结果集合 R
 - 5: **return** R
-

SQ 粗排时我们通过反量化计算 score 维护一个最小堆，保留分数最高的 Top- p 元素，因为这里计算的是内积，而非内积距离。粗排阶段的优化在 5.2 详细说明。

```

1 for (size_t i = 0; i < base_number; ++i) {
2     // 获取第 i 个向量的量化数据
3     const uint8_t* bq = base_q + i * vecdim;
4     // 计算量化向量点积
5     uint32_t qdot = dot_u8_neon_unroll32(bq, query_q.data(), vecdim);
6     // 利用反量化公式计算 score
7     float score = base_bias[i] + step2 * static_cast<float>(qdot);
8     // 当前向量编号
9     uint32_t id = static_cast<uint32_t>(i);
10    // 最小堆维护
11    if (coarse_pq.size() < top_p) {
12        coarse_pq.push(std::make_pair(score, id));
13    }
14    else if (score > coarse_pq.top().first) {
15        coarse_pq.pop();
16        coarse_pq.push(std::make_pair(score, id));
17    }
18 }

```

5.2 程序核心优化

SQ-SIMD 的优化重点有两个：一是量化空间中的粗排距离计算，二是粗排候选集大小 Top- p 的选择，而对于精排阶段的计算，我们选择复用 `InnerProductSIMDNeon_Unroll32` 函数进行计算。

首先对于量化空间下的粗排距离计算，我们首先针对量化空间下的 `uint8_t` 点积计算使用 NEON SIMD 指令集进行优化：

```

1 static inline uint32_t dot_u8_neon_unroll32(const uint8_t* a, const uint8_t* b,
2       size_t dim) {
3     assert(a != nullptr);
4     assert(b != nullptr);
5     assert(dim % 16 == 0);
6     // 采用四路累加器加速运算，一次计算 16 个 uint8
7     uint32x4_t acc0 = vdupq_n_u32(0);
8     uint32x4_t acc1 = vdupq_n_u32(0);
9     uint32x4_t acc2 = vdupq_n_u32(0);
10    uint32x4_t acc3 = vdupq_n_u32(0);
11    size_t i = 0;
12    // 32 路循环展开，每次处理 32 个 uint8
13    for (; i + 31 < dim; i += 32) {
14        // 使用 vld1q_u8 加载 uint8
15        uint8x16_t a0 = vld1q_u8(a + i);
16        uint8x16_t b0 = vld1q_u8(b + i);
17        uint8x16_t a1 = vld1q_u8(a + i + 16);
18        uint8x16_t b1 = vld1q_u8(b + i + 16);
19        // 防止溢出拆分成高 64 位和低 64 位分别计算
20        uint16x8_t p0_low = vmull_u8(vget_low_u8(a0), vget_low_u8(b0));
21        uint16x8_t p0_high = vmull_u8(vget_high_u8(a0), vget_high_u8(b0));
22        uint16x8_t p1_low = vmull_u8(vget_low_u8(a1), vget_low_u8(b1));
23        uint16x8_t p1_high = vmull_u8(vget_high_u8(a1), vget_high_u8(b1));
24        // 合并累加到 32 位累加器
25        acc0 = vpadalq_u16(acc0, p0_low);
26        acc1 = vpadalq_u16(acc1, p0_high);
27        acc2 = vpadalq_u16(acc2, p1_low);
28        acc3 = vpadalq_u16(acc3, p1_high);
29    }
30    // ... 处理剩余维度
31    uint32x4_t acc = vaddq_u32(vaddq_u32(acc0, acc1), vaddq_u32(acc2, acc3));
32    uint32_t tmp[4];
33    vst1q_u32(tmp, acc);
34    return tmp[0] + tmp[1] + tmp[2] + tmp[3];

```

在 SIMD 计算方面，由于 base 向量已经被压缩为 8-bit 数据，于是一个 NEON 寄存器一次加载更多维度的数据。我们让 NEON 寄存器一次加载 16 个 `uint8` 数据，这相比 `float32` 一次只能加载 4 个数据极大的提升了程序的效率，让 SQ 具有更高的数据并行度。

指令集方面，优化后的程序使用了 `vld1q_u8` 加载向量，使用 `vmull_u8` 和 `vaddq_u32` 执行并行乘法与并行加法，使用 `vpadalq_u16` 实现 16 位向量累加，指令优化后速度远快于普通循环。

同时在粗排阶段计算排序 **score** 时，直接使用公式逆推会涉及大量的重复运算。在 SQ 量化中，数据库向量的第 j 维可以近似表示为 $x_j \approx x_j^q \cdot \text{step} + \text{min_val}$ ，其中 x_j^q 为量化后的 uint8 数值，step 为量化步长，min_val 为全局最小值。我们分析反量化公式得到：

$$\text{score}(x, q) = \sum_{j=1}^d (x_j^q \cdot \text{step} + \text{min_val}) q_j = \text{step} \sum_{j=1}^d x_j^q q_j + \text{min_val} \sum_{j=1}^d q_j.$$

可以看到，第二项 $\text{min_val} \sum_{j=1}^d q_j$ 与数据库中的具体向量无关，只与当前查询向量有关。因此，我们在构建索引阶段为每个数据库向量预先计算 $\text{bias}_i = \text{min_val} \sum_{j=1}^d x_{i,j}$ ，从而减少粗排阶段的运算量，提高 SQ 粗排的执行效率。

5.3 实验结果呈现及分析

本实验代码中，SQ 的精排阶段直接复用 Flat-SIMD 的距离计算函数，这样可以减少量化误差对最终 recall 的影响。粗排阶段只负责减少需要进入精排的候选数量，因此 Top- p 是 SQ 方法中最关键的 trade-off 参数。当 p 较小时，精排计算量较少，但可能导致 latency 较低；当 p 较大时，更多候选进入精排，recall 提升，但精排开销增加，latency 也随之上升，因此我们需要选择一个合适的 p 确保 recall 和 latency 的平衡。对于参数 p 的选取，我测试了从 $p = 20$ 一直到 $p = 1000$ 的 6 组参数，SQ-SIMD 的性能如下图：

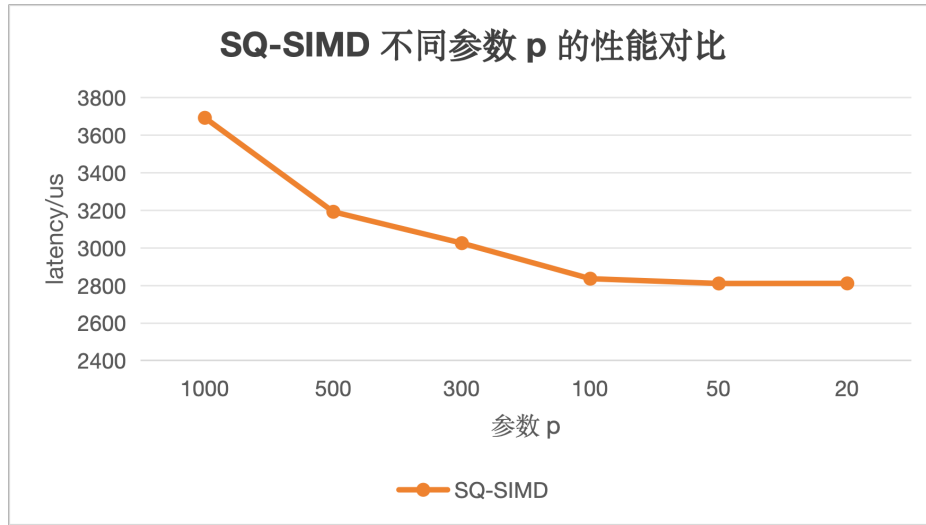


图 5.1: SQ-SIMD 的 latency-p 曲线

表 3: SQ-SIMD 最优参数 p

参数 p	1000	500	300	100	50	20
recall	0.99995	0.99995	0.99995	0.99995	0.99995	0.99995
latency/us	3693.16	3192.65	3025.64	2836.33	2810.88	2811.35

我发现对于当前的测试数据，上述 6 组取值的 recall 均为 0.99995，因此我们选择一个较小的 p 来减小精排阶段的计算量，当 $20 \leq p \leq 100$ 时，总体的 latency 在 2800 us 左右波动，主要受服务器影响，因此我们折中选择 $p = 50$ 作为最终参数。

我使用 perf 分析了 $p = 50$ 时程序的各项指标, 并与 Flat-Naive (下称 Naive) 和 Flat-SIMD-Unroll (下称 Flat-SIMD) 进行对比:

表 4: Flat-SIMD 和 SQ-SIMD 的性能对比

方法	recall	latency /us	cycles	instructions	cache-references	cache-misses
Flat-Naive	0.99995	17497.8	90156867167	118270999252	38834098348	499759194
Flat-SIMD	0.99995	5623.78	35119578104	19671108562	10033398146	170345304
SQ-SIMD	0.99995	2810.88	49414257323	90437048102	19527486643	147126514

如表所示, SQ-SIMD 的 average recall 仍保持在 0.99995, 与 Flat-SIMD 基本一致, 说明当 $p = 50$ 时, 量化误差没有明显影响最终 Top- k 结果。在查询延迟方面, SQ-SIMD 的 latency 为 2810.88 us, 相比 Flat-SIMD 的 5623.78 us 获得了约 2.00 倍加速, 相比 Flat-Naive 的 17497.8 us 获得了约 6.23 倍加速。其主要原因是 SQ 粗排阶段使用 uint8 量化向量代替 float32 原始向量进行扫描, 并且只对 Top- p 个候选执行原始空间精排, 从而减少了完整 float 向量距离计算的次数。

从 perf 指标看, SQ-SIMD 的 cycles 和 instructions 均高于 Flat-SIMD, 这是因为 SQ 额外引入了 query 量化、粗排、Top- p 候选维护和 rerank 等步骤。但 SQ-SIMD 的 IPC=1.83, 均高于之前的方法, 且 SQ-SIMD 在实际 latency 上更低, 这得益于优化后 CPU 单周期执行效率高、指令级并行更好。

缓存方面, SQ-SIMD 的 cache-misses 为 1.47×10^8 , 低于 Flat-SIMD 的 1.70×10^8 , 也明显低于 Flat-Naive 的 4.99×10^8 。这是因为 uint8 量化数据的数据体积仅为 float32 的四分之一, 粗排阶段能够减少内存访问量并提高缓存利用率。

综上, SQ-SIMD 进一步通过量化粗排减少原始向量精确计算和访存规模。在保持 recall 不变的前提下, SQ-SIMD 将平均查询延迟降至约 2.8 ms, 相比 Flat-SIMD 进一步提升约 2.00 倍, 相比原始串行版本提升约 6.23 倍, 说明 SQ 与 SIMD 结合能够有效改善大规模向量检索中的计算和访存瓶颈。

6 PQ-SIMD

6.1 程序实现思路

PQ (Product Quantization, 乘积量化) 是一种更进一步的向量压缩和近似距离计算方法。与 SQ 对每个维度独立量化不同, PQ 将原始 d 维向量划分为 M 个低维子向量, 然后在每个子空间内分别进行聚类, 得到 K_s 个聚类中心。对于一条原始向量 x , PQ 保存每个子空间中最近聚类中心的编号:

$$code(x) = (o_1, o_2, \dots, o_M), \quad o_m \in [0, K_s - 1].$$

这样, 一条 d 维浮点向量可以被压缩为 M 个整数编码。本次实验将原始向量切分为 4 个字段, 每个子段进行 256 类的聚类, 即 $d = 128, M = 4, K_s = 256$ 。在此情况下, 原始向量需要 $128 \times 4 = 512$ 字节, 而 PQ 编码只需要 4 字节, 压缩率很高。

本实验中的 PQ 查询阶段采用 ADC (非对称距离计算)。ADC 不对查询向量进行 PQ 编码, 而是保留查询向量的原始精度。查询到达时, 程序首先将查询向量划分为 M 个子向量, 并计算每个查询子向量与对应子空间中 K_s 个聚类中心之间的距离, 构建大小为 $M \times K_s$ 的查找表 LUT:

$$LUT[m][c] = \delta(q_m, center_{m,c}).$$

然后对每条 PQ 编码向量，根据其编码在 LUT 中查表并累加，即可得到近似距离：

$$\delta(q, x) \approx \sum_{m=0}^{M-1} LUT[m][code(x)_m].$$

完成粗排后，程序选出 Top- p 个候选，再使用原始向量和 Flat-SIMD 距离计算函数进行精排，得到最终 Top- k 结果。

PQ-SIMD 的整体查询流程如下：

Algorithm 2 PQ-SIMD 基于 ADC 的两阶段查询流程

Input: 查询向量 q , PQ 码本 C , PQ 编码数据库 $Codes$, 原始数据库 X , 候选数量 p , 返回数量 k

Output: Top- k 最近邻结果

- 1: 使用 Kmeans++ 进行 PQ 编码
 - 2: 构建查找表 $LUT[M][K_s]$
 - 3: 计算 ADC 距离粗排得到候选集合 C
 - 4: 复用 Flat-SIMD 精排得到结果集合 R
 - 5: **return** R
-

6.2 程序核心优化

PQ-SIMD 的优化可以分为三个部分：PQ 编码阶段优化、LUT 构建优化和查表累加优化。

在 PQ 编码阶段，我们使用 KMeans++，即平均初始化聚类中心 `centroid`，聚类常使用 L2 Distance，所以针对于 L2 距离的计算使用了 SIMD 指令集优化：

```

1 static inline float l2_distance_simd(const float *a, const float *b, size_t dim) {
2     size_t i = 0;
3     // 采用二路累加器加速运算
4     float32x4_t acc0 = vdupq_n_f32(0.0f);
5     float32x4_t acc1 = vdupq_n_f32(0.0f);
6     // 8 路循环展开，一次处理 8 个 float
7     for (; i + 8 <= dim; i += 8) {
8         // 加载 a, b 的前 4 个 float，使用 vsubq_f32 并行计算差值
9         float32x4_t a0 = vld1q_f32(a + i);
10        float32x4_t b0 = vld1q_f32(b + i);
11        float32x4_t d0 = vsubq_f32(a0, b0);
12        // 同理加载第 2 组
13        ... ..
14        // fma 指令并行计算平方和
15        acc0 = vfmaq_f32(acc0, d0, d0);
16        acc1 = vfmaq_f32(acc1, d1, d1);
17    }
18    // 合并累加器
19    float32x4_t acc = vaddq_f32(acc0, acc1);
20    // 横向求和
21    float dis = vaddvq_f32(acc);
22    // 处理剩余元素
23    ... ..
24    return dis;
25 }
```

函数使用 `vdupq_n_f32` 加上循环展开一次处理 8 位浮点数，并且使用 `vfmaq` 指令融合乘加运算，最后使用 `vaddvq_f32` 自动规约求和，效率极高，虽然构建 PQ 索引是离线过程，但是此函数也可用于查询阶段的 LUT 构建。LUT 的构建本质也是距离的计算，我们将其向量化后计算，极大提升了查询的效率。

针对 LUT 表的构建，为了减小函数调用开销与循环判断开销，我们手工展开 `centroid` 进行并行处理，这样有效减小了查找所需的 latency，并且仿照 Flat-SIMD 对循环体内的运算采用了指令优化，代码如下：

```

1 // 手工展开 4 个 centroid 构建 LUT 索引
2 void build_lut(const float *query, std::vector<float> &lut) const {
3     ... ..
4     // 遍历每个子空间 m
5     for (size_t m = 0; m < M; ++m) {
6         const float *qsub = query + m * dsub;
7         float *lut_m = lut.data() + m * Ks;
8         size_t c = 0;
9         // 一次计算 4 个 centroid 的距离
10        for (; c + 4 <= Ks; c += 4) {
11            const float *center0 = centroid_ptr(m, c + 0);
12            const float *center1 = centroid_ptr(m, c + 1);
13            const float *center2 = centroid_ptr(m, c + 2);
14            const float *center3 = centroid_ptr(m, c + 3);
15            // 为 4 个聚类中心分别初始化累加器，并行计算 4 个 float 的平方和
16            float32x4_t acc0 = vdupq_n_f32(0.0f);
17            float32x4_t acc1 = vdupq_n_f32(0.0f);
18            float32x4_t acc2 = vdupq_n_f32(0.0f);
19            float32x4_t acc3 = vdupq_n_f32(0.0f);
20            // 遍历子向量内部维度，四路展开
21            size_t j = 0;
22            for (; j + 4 <= dsub; j += 4) {
23                float32x4_t qv = vld1q_f32(qsub + j);
24                float32x4_t c0 = vld1q_f32(center0 + j);
25                ... ..
26                float32x4_t d0 = vsubq_f32(qv, c0);
27                ... ..
28                acc0 = vfmaq_f32(acc0, d0, d0);
29                ... ..
30            }
31            // 对每个聚类中心的累加器横向求和，得到最终 L2 距离平方
32            float dis0 = vaddvq_f32(acc0);
33            float dis1 = vaddvq_f32(acc1);
34            float dis2 = vaddvq_f32(acc2);
35            float dis3 = vaddvq_f32(acc3);
36            // 写入 LUT 表
37            ... ..
38        }
39    }

```

查表累加是 PQ 查询阶段的核心。对于每条 base 向量，程序只需要执行 M 次查表和 M 次左右的加法，不再需要访问完整的 d 维 float 向量。因此，PQ 粗排阶段的理论计算复杂度从 Flat 的 $O(Nd)$ 降低为 $O(NM)$ 。当 $M \ll d$ 时，计算量大幅减少。同时，PQ 编码本身非常紧凑，也显著降低了数据库的访存量。针对使用 ADC 距离进行粗排进行了循环体的优化，核心部分如下：

```

1      size_t i = 0;
2      // 一次处理 4 个 base 向量
3      for (; i + 4 <= n; i += 4)
4          // 获取连续 4 个库向量的 PQ 编码指针
5          const uint8_t *code0 = codes.data() + (i + 0) * M;
6          const uint8_t *code1 = codes.data() + (i + 1) * M;
7          const uint8_t *code2 = codes.data() + (i + 2) * M;
8          const uint8_t *code3 = codes.data() + (i + 3) * M;
9          // 初始化 4 个向量的 PQ 距离
10         float dis0 = 0.0f;
11         float dis1 = 0.0f;
12         float dis2 = 0.0f;
13         float dis3 = 0.0f;
14         // 遍历所有 M 个 PQ 子空间，累加每个子空间的查表距离
15         for (size_t m = 0; m < M; ++m) {
16             const float *lut_m = lut.data() + m * Ks;
17             dis0 += lut_m[code0[m]];
18             dis1 += lut_m[code1[m]];
19             dis2 += lut_m[code2[m]];
20             dis3 += lut_m[code3[m]];
21         }
22         // 生成 4 个向量的原始 ID
23         uint32_t id0 = static_cast<uint32_t>(i + 0);
24         uint32_t id1 = static_cast<uint32_t>(i + 1);
25         uint32_t id2 = static_cast<uint32_t>(i + 2);
26         uint32_t id3 = static_cast<uint32_t>(i + 3);
27         // 维护最大堆
28         if (heap.size() < top_p) {
29             heap.push(std::make_pair(dis0, id0));
30         }
31         else if (dis0 < heap.top().first) {
32             heap.pop();
33             heap.push(std::make_pair(dis0, id0));
34         }
35         ... ..

```

由于查表累加访问 LUT 时具有一定随机性，访存局部性较弱，其随机索引导致相比连续数组上的内积计算，查表操作更难直接向量化。我们通过展开循环体，一次处理 4 个 base 向量，通过批量处理多个向量来摊薄访存开销。

6.3 实验结果呈现及分析

PQ 中 Top- p 的作用与 SQ 类似。较小的 p 可以减少精排开销，但可能由于 PQ 量化误差导致 recall 下降；较大的 p 可以提高 recall，但会增加原始向量精排成本。因此 PQ-SIMD 的实验结果也需要围绕 latency-recall trade-off 进行分析。我首先对比了不同 Top- p 参数下 PQ-SIMD 的性能，帮助我们确定最优的参数 p 。

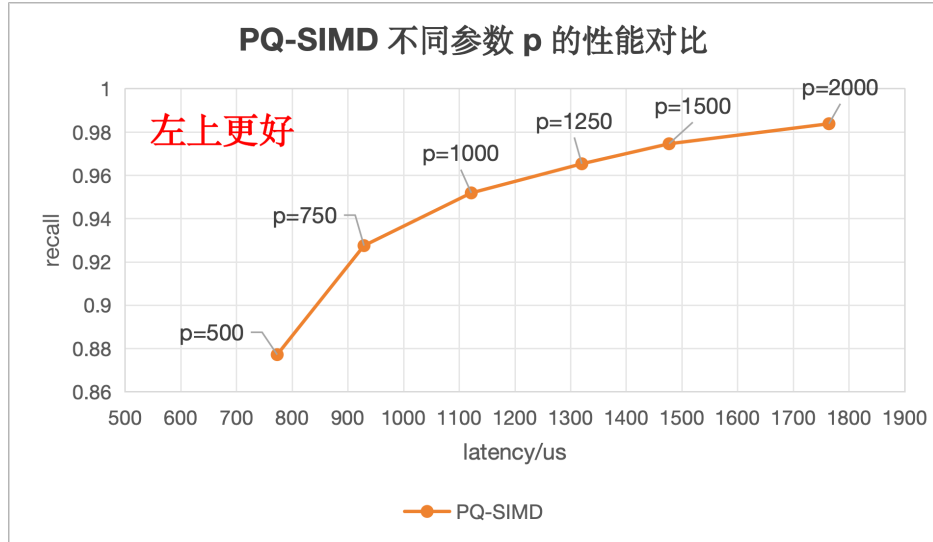


图 6.2: PQ-SIMD 的 Latency-Recall Trade-off 曲线

根据 trade-off 曲线，我们希望选择保持高 recall 的同时还具备低 latency 的参数 p ，最终我认为有两组参数都具备较优的性能，如表 4 所示。如果需要更高的精准性 (recall > 0.95)，我们可以选择 $p = 1000$ ，如果需要更快的速度，我们可以选择 $p = 750$ ，此时的 recall 也大于 0.9。在这里我选择 $p = 1000$ 进行后续的分析。

表 5: PQ-SIMD 最优参数 p

参数 p	2000	1500	1250	1000	750	500
recall	0.983852	0.974603	0.965354	0.951904	0.927554	0.877204
latency/us	1763.75	1476.68	1319.97	1121.47	928.869	773.031

表 6: PQ-SIMD, SQ-SIMD, Flat-SIMD 性能对比

方法	recall	latency / us	加速比 (对比 Flat)
Flat-SIMD	0.99995	5623.78	-
SQ-SIMD	0.99995	2810.88	2.00
PQ-SIMD	0.951904	1121.47	5.01

从实验结果可以看出，PQ-SIMD 在 $p = 1000$ 时取得了 0.951904 的 recall，虽然低于 Flat-SIMD 和 SQ-SIMD 的 0.99995，但仍然保持在较高水平。与此同时，PQ-SIMD 的 latency 仅为 1121.47 us，相比 Flat-SIMD 的 5623.78 us 获得了约 5.01 倍加速；相比 SQ-SIMD 的 2810.88 us，也获得了约 2.51 倍加速。这说明 PQ-SIMD 在牺牲一定 recall 的前提下，显著降低了查询延迟，体现出更强的速度优势。

与 SQ-SIMD 相比, PQ-SIMD 将向量划分为多个子空间, 并为每个子空间预先训练聚类中心, 在离线训练阶段完成了绝大多数计算工作, 不需要像 SQ-SIMD 一样在量化空间中对每个维度进行乘加运算, 只需要进行简单的查表累加运算即可, 大幅减少了粗排阶段的计算量。

除此之外, 我通过 perf 工具进一步分析了 PQ-SIMD 实现的具体细节:

表 7: PQ-SIMD 性能分析

cycles	instructions	cache-references	cache-miss	IPC	miss rate
37105316009	76491486046	18850574626	48147108	2.06	0.255%

从 perf 数据看, PQ-SIMD 的 IPC 指标在三种方法上最高, 这表明 PQ-SIMD 的流水线利用率较高, 单位周期内能够完成更多指令。其原因在于 ADC 查表累加主要由整数索引访问、浮点加法和简单循环构成, 计算依赖关系相对较短, 且不需要像 Flat-SIMD 那样对完整 float32 向量进行大规模连续乘加计算, 因此处理器执行效率更高。且 PQ-SIMD 的总体循环数与指令数也显著低于 SQ-SIMD, 印证了 PQ 方法在粗排上计算的高效。

缓存行为方面, PQ-SIMD 的 miss rate 仅为 0.255%, 明显低于 SQ-SIMD 的 0.753% 和 Flat-SIMD 的 1.698%, 这说明 PQ-SIMD 很好的改善了缓存访问局部性。我们使用的 LUT 表大小通常为 $M \times K_s$, 可以较好地保存在 cache 中, 并且我们批处理向量进一步摊薄了仿存开销, 因此查表累加的 cache miss 很低。

针对 PQ-SIMD recall 下降的问题, 由于聚类训练过程中难免会引入量化误差, 使得我们计算的 ADC 距离和真实距离之间存在差距, 可以通过提高 p , 增加 codebooks 的训练轮次, 修改初始划分个数 M 等等改善, 本次实验已经做了比较好的 trade-off, 采用 $p = 1000, M = 4, d_{sub} = 24, train_iters = 10$ 。

综上, PQ-SIMD 通过更紧凑的编码, 更高效的预处理计算, 以及对于查表累加的优化, 实现了明显的速度提升, 在稍微牺牲 recall 的情况下, PQ-SIMD 在 $p = 1000$ 时将 latency 降低到 1121.47 us, 相比 SQ-SIMD 获得约 2.51 倍加速, 相比 Flat-SIMD 获得约 5.01 倍加速, 相比 Naive 方法获得了约 15.60 倍加速。

7 汇编代码分析

为了进一步分析不同优化策略产生性能差异的原因, 本实验使用 perf 对程序运行过程中的硬件事件进行采样。同时借助 gogbolt 分析生成的汇编代码, 进一步分析性能差异的原因。我们发现“距离计算”在本次实验任务中起到了举足轻重的作用, 于是我们重点分析“距离计算”有关的函数代码。

Listing 1: 手写 SIMD 版本核心循环

```

1 .L5:
2     ldr    q4, [x1, -16] @ 从 query 向量加载 4 个 float
3     ldp    q5, q3, [x0] @ 从 base 向量加载 8 个 float
4     add    x0, x0, 32
5     ldr    q2, [x1], 32
6     fmla   v1.4s, v5.4s, v4.4s @ 4 个 float 并行计算向量乘加
7     fmla   v0.4s, v3.4s, v2.4s
8     cmp    x2, x0
9     bne    .L5

```

Listing 2: 自动向量化版本核心循环

```
1 .L6:
2     ldr    q2, [x0, x3] @ 加载 4 个 float
3     ldr    q1, [x1, x3]
4     add    x3, x3, 16
5     fmla   v0.4s, v2.4s, v1.4s @ 4 个 float 并行计算向量乘加
6     cmp    x3, x4
7     bne    .L6
```

手写 SIMD 和编译器 -O3 自动向量化都利用了 ARM NEON 的向量寄存器和 fmla 指令，将原本逐元素执行的浮点乘加运算转化为 4 路并行计算。二者本质上都实现了数据级并行，但是，手写 SIMD 版本进一步进行了循环展开，每轮循环处理 8 个浮点数，并使用两个独立的向量累加器分别累加前后两组数据，这种方式进一步减少了循环控制开销。相比之下，编译器 -O3 自动向量化生成的代码更保守，每轮循环只处理 4 个浮点数，并主要使用单一累加器进行累加。

同时在访存方面，自动向量化版本使用的是基址加偏移寻址 `add x0, x0, 32`，保留了原始指针 `x0` 和 `x1` 不变，通过偏移量 `x3` 控制当前访问位置；而手写 SIMD 版本则使用了指针递增方式，`ldr q2, [x1], 32` 使用了后索引寻址，即加载完成后自动更新指针。这样可以减少显式地址计算指令，使循环更紧凑。

因此，在本次实验中，如果依赖编译器自动向量化进行大量计算的话，会因为其保守的策略而在性能上不及手写的 SIMD 指令优化程序，但自动向量化的版本更加稳健安全，具有更好的可维护性。

8 实验总结与思考

本次实验围绕 ANNS 向量检索任务，依次实现并分析了 Flat-SIMD、SQ-SIMD 和 PQ-SIMD 三类优化方法。我们主要从两方面入手进行优化：一是加速单次距离计算本身的速度，二是减少检索需要访问的点的个数。我们主要通过 SIMD 指令优化来加速距离计算，通过 NEON 向量指令将浮点数向量化计算，提升程序并行计算能力；通过 SQ、PQ 等量化方法减小访存压力，通过压缩向量表示和减少候选数量降低访存与计算规模，从而提升检索速度。

Flat-SIMD 是本次实验的基础。它重点优化了内层距离计算，显著减少了 base 向量逐维计算的开销，同时也是本次实验后续 SQ 和 PQ 精排阶段的基础。SQ-SIMD 在 Flat 的基础上引入标量量化，将 float32 向量压缩为低精度整数表示；PQ-SIMD 则进一步利用子空间量化和查找表累加减少粗排阶段的计算量。同时，SQ 和 PQ 都要求我们进行比较严谨的 trade-off 分析，以选取最优的参数获得最好的效果。

总体而言，本次实验加深了我对 ANNS 这一任务的理解，强化了我对于并行计算的应用能力，也更加熟练的掌握了 SIMD 指令的运用。同时通过对一些 profiling 工具的使用，我对于程序内部加速运行的逻辑也有了更加直观的理解。